



AquaZero

A I W E L L N E S S A S S I S T A N T

NIT3004 IT CAPSTONE PROJECT 2

Development Diary

Dated work records, meeting minutes and decision log

Project	AquaZeroFit, an AI-powered wellness platform (web and Telegram Mini App)
Team members	Babatundji Williams-Fulwood (s8138393), Eric La, Victor Hong
Supervisor	Chenhao Xu
Group	Group [number], Victoria University, Melbourne
Period covered	Monday 27 July 2026 to Friday 31 July 2026
Meetings recorded	5 (one per working day)
Document status	Issued for assessment
Companion document	AQF-17 Progress Report

Contents

Right-click the table of contents and choose "Update Field" to fill in page numbers.

How This Diary Works	3
Rules the team follows for this diary	3
Project references.....	3
Verification status at the end of the reporting period	3
1. Project Setup and Code Structure	4
2. Shared Rules Package	5
3. Accounts, Sign-in and Profile.....	6
4. Getting Exercise and Food Data In	8
5. Food Logging and the Daily Dashboard.....	9
6. Meal Photo Recognition	10
7. The AI Layer	12
8. The Coach Assistant and Personal Memory.....	13
9. Training Plans and Workouts	15
10. Progress Tracking	15
11. Interface Design System and Telegram Support.....	16
12. Data Storage and Deployment Readiness.....	17
13. Meeting Records	19
Meeting 1: Project kickoff	19
Meeting 2: Sign-in and data ingestion	20
Meeting 3: Nutrition loop and AI rules	21
Meeting 4: Coach, memory and training	22
Meeting 5: Progress review and 80% assessment	22
14. Design and Style Constants	23
15. Data Storage Design	24

How This Diary Works

This is the team's development diary for AquaZeroFit. It is the supporting evidence behind the Progress Report (AQF-17). The Progress Report says what state the project is in. This diary says how it got there, day by day, and who did each piece.

Reading it as a non-technical reader: each section covers one part of the app. Each section opens with a short plain-English explanation of what that part does and why it matters, then gives a dated table of the actual work. You can read only the explanations and still follow the whole project.

Reading it as a developer: each section names the files and modules touched, so this diary and the repository history confirm each other.

Rules the team follows for this diary

- 1. One table per part of the system, one row per working session.** Each row records the date, the member, and the specific change made.
- 2. Entries are written the same day.** A diary written from memory at the end is not evidence, and reads like it.
- 3. Every entry names the files or modules touched.** This lets a reader cross-check any claim against the code.
- 4. The diary is updated after every meeting,** so meeting decisions and the work that followed sit next to each other.
- 5. Problems are recorded, not hidden.** Several entries below describe things that were wrong and had to be fixed. That is the point of a diary.

Project references

Item	Location
Application repository	Team GitHub (private), a single organised codebase
Main parts	apps/api (the server), apps/web (what the user sees), packages/shared (rules used by both)
AI instruction files	prompts/ P-01 to P-11, each version-numbered
Safety test sets	evals/, run with <code>npm run eval</code>
Document set	docs/specs/, AQF-01 to AQF-22
Design references	design/figma/ (16 screens), design/brand/
Verification commands	<code>npm run typecheck</code> , <code>npm test</code> , <code>npm run eval</code>

Verification status at the end of the reporting period

Check	Result on Friday 31 July 2026
Type safety across the whole codebase	0 errors
Automated tests	431 passing across 35 files (427 server, 4 interface)
Safety evaluation gate	Passed, zero critical misses, zero allergen violations
Continuous integration	Configured to run all of the above automatically on every change

1. Project Setup and Code Structure

What this is: before writing any features, the team had to decide how the code would be organised. Getting this wrong makes everything afterwards slower and makes it impossible to tell who wrote what.

Files and areas: `package.json`, `tsconfig.base.json`, `.gitignore`, `.env.example`, `apps/api/src/platform/config.ts`, `.replit`, `.github/workflows/ci.yml`

Date	Member	Record
27/07/2026	Babatundji	Created the project as three connected parts in one repository: the server, the user interface, and a shared package holding rules both use. Set the strict language settings that every part inherits, so mistakes are caught automatically rather than by review.
27/07/2026	Babatundji	Agreed and documented the naming rule for the server: every feature area gets its own folder with a routing file and a logic file, and anything shared sits in a separate platform folder. This is why a reader can find any feature without asking.
27/07/2026	Babatundji	Wrote the central configuration file. All settings come from the environment with safe defaults for local development, and the app refuses to start in

Date	Member	Record
		production if a real security key is missing, rather than quietly running with a development key.
27/07/2026	Victor	Wrote the environment template documenting every setting with an explanatory comment, and confirmed the real settings file is excluded from the repository so secret keys can never be committed by accident.
31/07/2026	Babatundji	Reorganised the repository to hand-over quality: documents into docs/, design assets into design/, document tooling into tools/ with its own dependency list so the document generator never ships inside the deployed app. The top level went from 20 items to 13.
31/07/2026	Babatundji	Recorded an important constraint in the README: the prompts/ and evals/ folders must stay at the top level, because the code finds prompt files by searching upward from its own location. Moving them would silently break the AI with no error message, which is worse than a crash.
31/07/2026	Babatundji	Added the automated build pipeline so every change pushed to the repository automatically runs the type check, the full test suite, the safety gate and the build.

2. Shared Rules Package

What this is: the definition of what valid data looks like, for example what counts as a sensible weight or a valid email. Writing this once and sharing it means the app and the server can never disagree about the rules.

Files: packages/shared/src/types.ts, schemas.ts, constants.ts, errors.ts

Date	Member	Record
27/07/2026	Babatundji	Defined the shared data shapes and validation rules used by both the interface and the server. This means a form on screen and the server behind it enforce exactly the same limits, with no duplicated logic to drift apart.
27/07/2026	Babatundji	Defined the single list of error types the whole system uses, so every error the user sees has a consistent shape and the interface always knows how to display it.
28/07/2026	Victor	Added the safe-range limits for profile inputs, plus the wellness disclaimer text and the crisis support message, so safety wording lives in one place and cannot drift between screens.

3. Accounts, Sign-in and Profile

What this is: letting a user create an account and prove who they are, then keeping them signed in safely. This also covers calculating their daily calorie target.

Files: apps/api/src/modules/auth/service.ts, router.ts, telegram.ts, apps/api/src/platform/auth.ts, apps/api/src/modules/me/service.ts, targets.ts

Date	Member	Record
28/07/2026	Babatundji	Built sign-up and sign-in. Passwords are never stored, only an irreversible scrambled version, so even someone with full database access cannot read them.

Date	Member	Record
28/07/2026	Babatundji	Built the session system. A short-lived pass lasts 15 minutes and a longer renewal pass can be exchanged for a new one. Each renewal pass works only once. If an old one is reused, which is what happens when a pass has been stolen, every session in that chain is cancelled immediately.
28/07/2026	Babatundji	Built Telegram sign-in. Telegram signs the data it sends, and the server checks that signature using a comparison method that takes the same amount of time whether it matches or not, so an attacker cannot learn anything from timing. Also checks the data is recent, so old captured sign-in data cannot be replayed. Wrote three test cases: valid, tampered, and expired.
28/07/2026	Babatundji	Built the calorie target calculator, including the safety floor. If someone's inputs would produce a dangerously low target, the number is raised to a safe minimum and a visible notice explains why. Tests cover every path including the clamped one.
28/07/2026	Eric	Built the welcome, sign-in and multi-step onboarding screens, with validation messages appearing as the user types, using the shared rules so screen and server agree.
29/07/2026	Victor	Wrote the sign-in test suite (10 tests): registration, sign-in, wrong password, and the stolen-pass scenario. Confirmed

Date	Member	Record
		that a wrong password and an unknown email give an identical error, so an attacker cannot discover which email addresses have accounts.
30/07/2026	Babatundji	Built the password reset flow: a single-use code that expires after 30 minutes and cancels all existing sessions when used. Recorded openly that no email service is connected yet, so the code cannot actually be delivered to a user. This is listed in the Progress Report as outstanding rather than being quietly left.

4. Getting Exercise and Food Data In

What this is: the app needs a library of exercises and a database of foods. Both come from open projects, which means the team must credit them correctly or the project breaks the licence terms.

Files: `apps/api/src/data/wger/importer.ts`, `apps/api/src/modules/foods/service.ts`, `offImporter.ts`, `content/ATTRIBUTION.md`

Date	Member	Record
28/07/2026	Victor	Researched licences before writing any import code, so the team did not build on data it could not legally use. Recorded the findings and which sources were acceptable.
28/07/2026	Victor	Imported the exercise library from the open wger project, 828 exercises, keeping the licence and the original author on every single record. This matters: the licence requires credit per item, so storing it once for the whole set would not satisfy the terms.

Date	Member	Record
28/07/2026	Victor	Added the credit line shown in the app next to each exercise, naming the author, the licence and the source.
29/07/2026	Victor	Connected the Open Food Facts database for barcode lookups. Its licence has different obligations from the exercise data, so those records are kept in a completely separate area of the database and never mixed with the curated content.
29/07/2026	Victor	Added a cross-check that catches bad nutrition data, comparing the stated calorie figure against the calories implied by the protein, carbohydrate and fat values. Crowd-sourced data contains errors, and this rejects records that contradict themselves.
30/07/2026	Victor	Started replacing placeholder exercise artwork with curated images, and recorded the current coverage so progress is measurable.

5. Food Logging and the Daily Dashboard

What this is: the core daily loop. The user records what they ate, and sees how they are tracking against their target.

Files: `apps/api/src/modules/logs/service.ts`, `apps/api/src/modules/analytics/router.ts`, `apps/web/src/pages/nutrition/`, `apps/web/src/pages/dashboard/`

Date	Member	Record
29/07/2026	Babatundji	Built meal logging, plus water and weight logging on the same pattern. Two details that matter: the day boundary is calculated in the user's own

Date	Member	Record
		timezone, so a late meal lands on the correct day; and a repeated request caused by a flaky connection cannot create a duplicate entry.
29/07/2026	Babatundji	Built the daily totals calculation that powers the dashboard.
29/07/2026	Eric	Built the nutrition screen: a remaining-calories ring, macronutrient progress bars, and meals grouped by type. Built the dashboard cards including one-tap water logging and a weight trend line.
29/07/2026	Eric	Added loading, empty and error states to every card. A first-time user with no data sees encouraging guidance rather than a blank screen, and a failed request shows a retry option rather than nothing.
29/07/2026	Victor	Wrote 12 tests confirming the dashboard totals exactly equal the sum of the underlying records, including days that cross a timezone boundary.

6. Meal Photo Recognition

What this is: the headline feature. The user photographs a meal and the app works out what is on the plate.

Files: `apps/api/src/modules/vision/router.ts`,
`apps/web/src/pages/nutrition/CaptureMeal.tsx`, `AnalysisResults.tsx`

Date	Member	Record
29/07/2026	Babatundji	Built the pipeline: the photo is uploaded, analysed in the background, then shown to the user for confirmation before

Date	Member	Record
		anything is saved. The AI only identifies the foods. The calories and macronutrients are then looked up in the nutrition database and multiplied by the portion size in ordinary code. This means the same photo always produces the same numbers, and the arithmetic can be checked by hand.
29/07/2026	Eric	Built the capture and results screens: camera or file selection, a confidence score shown per identified item, portion adjustment, and the confirm step.
31/07/2026	Babatundji	Privacy fix. Photographs from a phone contain hidden data including the exact GPS coordinates where the picture was taken. Storing that alongside health records would have attached a home address to someone's dietary information. Every uploaded photo is now rebuilt from its pixels on arrival, which discards all hidden data. The rebuild also acts as the real file check: only genuine images survive it, so a disguised file cannot be uploaded.
31/07/2026	Babatundji	Upgraded the file-upload library after finding the version in use was no longer supported and carried a known flaw that let any signed-in user crash the server with a malformed upload.
31/07/2026	Victor	Wrote the upload security tests: built a test photograph containing GPS coordinates,

Date	Member	Record
		uploaded it, and confirmed the stored file contains no location data while the image itself is intact. Also confirmed that a non-image file pretending to be a photo is rejected cleanly.

7. The AI Layer

What this is: everything that talks to an AI model. Built as one controlled gateway rather than scattered calls, so safety and cost can be enforced in a single place.

Files: apps/api/src/modules/ai/gateway.ts, guardrails.ts, prompts.ts, creditLedger.ts, prompts/P-01 to P-11, evals/

Date	Member	Record
29/07/2026	Babatundji	Built the AI gateway. Every AI request in the app goes through this one place, sorted into task types such as chat, photo analysis and plan generation. If no AI service is connected, a built-in offline engine answers instead, so the whole app works with no paid account. That engine always gives the same answer for the same input, which is what makes automated testing possible.
29/07/2026	Babatundji	Wrote the AI instruction files P-01 to P-09, each version-numbered with a defined output format, so a change to how the AI behaves is a tracked change rather than an untraceable edit.
29/07/2026	Babatundji	Built the safety checks that run before and after every AI request, and the per-user credit system that caps how much AI

Date	Member	Record
		usage any single account can consume.
29/07/2026	Victor	Built the three safety test sets: 43 adversarial coach questions, 12 allergen cases, and 15 training plan cases. Wired them to a single command so the whole set can be run in seconds. The command reports failure if any critical case is missed, which makes it a gate rather than a report.
30/07/2026	Babatundji	Added the memory instruction files P-10 and P-11 and connected them to the conversation flow.
31/07/2026	Babatundji	Reliability work, after finding the AI layer could hang for around 100 seconds in the worst case. Added: automatic retry for temporary failures, an increasing wait between attempts so a struggling service is not overwhelmed, a rule that stops contacting a provider that keeps failing, and an overall 12-second limit for the whole chain. Also added a flag marking any answer that came from the offline engine after a real failure, so the app can tell the difference between a genuine AI answer and a fallback.

8. The Coach Assistant and Personal Memory

What this is: the chat assistant, and its ability to remember things about the user between conversations.

Files: apps/api/src/modules/chat/, apps/api/src/modules/memory/, apps/web/src/pages/coach/Coach.tsx, apps/web/src/pages/settings/Memory.tsx

Date	Member	Record
30/07/2026	Babatundji	Built the coach chat with answers streaming in as they are generated, rather than the user staring at a spinner. Every request passes through a fixed sequence of checks: confirm who the user is, check they are not sending too many requests, check they have credit, check the question is safe, call the AI, check the answer is safe, then reply.
30/07/2026	Babatundji	Built the personal memory system. The AI can suggest facts about the user, for example a dietary preference, but it can never confirm one itself. Suggested facts sit in a pending list until the user approves them. Every memory feature is switched off unless the user has explicitly given consent, and the app returns a clear refusal rather than silently working without it.
30/07/2026	Eric	Built the coach screen with a permanent visible notice that this is an AI assistant and not medical advice. Built the memory settings screen: pending facts with approve and reject buttons, the consent switch, and a calm explanatory state when consent is off rather than an error.
30/07/2026	Victor	Wrote the privacy isolation suite: 24 tests that sign in as one user and attempt to read or affect another user's records across every part of the app. All are correctly refused. Also 13 tests covering memory,

Date	Member	Record
		including what happens when a user withdraws consent.

9. Training Plans and Workouts

What this is: generating a personalised home workout plan and guiding the user through each session.

Files: apps/api/src/modules/plans/service.ts, progression.ts,
apps/api/src/modules/workouts/, apps/web/src/pages/training/

Date	Member	Record
30/07/2026	Babatundji	Built plan generation and the progression rules that increase difficulty as the user improves, with upper limits so the increase can never become unsafe. Added strength statistics estimated from the sets the user completes.
30/07/2026	Eric	Built the workout library with filters and equipment tags, the credit line for each exercise, and the workout detail screen with a guided session mode that walks the user through set by set.
30/07/2026	Victor	Wrote 22 progression tests and 17 training tests, including checks that the difficulty increase respects its safety limits.

10. Progress Tracking

What this is: showing the user how they are doing over time, which is what keeps people using a wellness app.

Files: apps/api/src/modules/progress/, apps/web/src/pages/progress/

Date	Member	Record
30/07/2026	Babatundji	Built the trend calculations for weight and calories, and the achievement rules.
30/07/2026	Eric	Built the progress screen: weight trend against the goal line, calorie trend, training history and achievements, plus the weight logging flow.

11. Interface Design System and Telegram Support

What this is: the shared visual language of the app, and the work needed for it to run properly inside Telegram.

Files: `apps/web/src/components/ui/`, `apps/web/src/styles/index.css`, `apps/web/src/lib/telegram.ts`, `design/figma/`

Date	Member	Record
27/07/2026	Eric	Translated the 16 design reference screens into a reusable set of building blocks: cards, buttons, tags, progress rings, metric tiles, notifications and loading placeholders. Building these first meant every later screen was assembled rather than drawn from scratch.
28/07/2026	Eric	Built the app frame: bottom navigation with tactile feedback, the header, and the animated background, which switches itself off for users who have asked their device to reduce motion.
30/07/2026	Eric	Added Telegram support: the app announces itself to Telegram on start, expands to full height, adopts the user's Telegram colour theme, and signs the user in automatically from the data Telegram

Date	Member	Record
		provides. If it is opened outside Telegram, all of this quietly does nothing and normal email sign-in is used.
31/07/2026	Eric	Confirmed each screen loads only its own code rather than the whole app at once, which keeps the initial load fast. Recorded outstanding polish: Telegram's own back button, on-screen keyboard handling, and a check that all tap targets are large enough.

12. Data Storage and Deployment Readiness

What this is: where the app's information actually lives, and what is required to put the app on the internet. This section contains the most significant problem found during the week.

Files: apps/api/src/platform/store.ts, pgStore.ts, apps/api/src/index.ts, apps/api/src/app.ts, .replit

Date	Member	Record
27/07/2026	Babatundji	Built the initial storage system, keeping data in memory for speed and saving it to files. Writes are queued so two simultaneous saves cannot corrupt a file, and a damaged file is set aside rather than crashing the app. Suitable for development, and always intended to be replaced before going live.
31/07/2026	Babatundji	Critical finding. Confirmed from the hosting provider's own documentation that a published app's file storage "is not persistent and resets every time you publish". Every user account, weight record, meal

Date	Member	Record
		log and conversation would have been erased on every update. This would not have shown up in development, because nothing is published during development. It would have appeared the first time the live app was updated, in front of real users.
31/07/2026	Babatundji	Rebuilt the storage layer in response. Data is now written through to a proper database that sits outside the app and is unaffected by updates. The design deliberately kept the rest of the application unchanged: converting all 24 affected files to a different style of database access would have taken roughly a week the team did not have, and would have risked breaking working features. Recorded the trade-off openly: this design is safe for one server, and running several copies at once would need further work, so the hosting is configured for a single server.
31/07/2026	Babatundji	Made the server also deliver the user interface, so the whole product runs on one address with one security certificate and no cross-origin complications. Added a safeguard, covered by a test, that stops the interface from accidentally intercepting data requests.
31/07/2026	Babatundji	Security headers, with one Telegram-specific detail. The standard way to stop other sites embedding your app would have broken the Telegram Mini App, because Telegram

Date	Member	Record
		legitimately embeds it. Used the modern equivalent that can name Telegram as an allowed exception, and deliberately did not send the older header that would have overridden it.
31/07/2026	Babatundji	Further production hardening: correct handling of the user's real network address behind the host, which if wrong would have made one user's activity count against everyone; separate health and readiness checks placed where they cannot be blocked; a clean shutdown that finishes in-flight requests and saves pending data; and stricter startup checks that refuse to launch with unsafe settings.
31/07/2026	Babatundji	Wrote the hosting configuration and the deployment guide covering publishing and connecting the team's domain name.
31/07/2026	Victor	Ran the full verification after integration: 431 tests passing across 35 files, type check clean, safety gate passed. Captured the live end-to-end evidence used in the Progress Report.

13. Meeting Records

Meeting 1: Project kickoff

Field	Detail
Project	AquaZeroFit
Date and time	Monday 27 July 2026, 10:00

Field	Detail
Location	Discord
Present	Babatundji Williams-Fulwood, Eric La, Victor Hong
Absent	None
Objective	Form the team, agree scope and roles, start the codebase

Discussion and decisions

- 1. Scope agreed** from the project brief: nutrition tracking, home training, and an AI coach, delivered as both a website and a Telegram Mini App.
- 2. Roles assigned.** Babatundji: architecture, server, AI and security. Eric: the entire user interface and design system. Victor: data ingestion, quality and verification evidence. The team chose bounded individual ownership over shared ownership, specifically so each member can demonstrate and explain their own work at the review.
- 3. Working agreement.** Daily meeting. A task counts as done only when it can be demonstrated, explained and verified. The diary is updated the same day.
- 4. Technical decisions.** One language across the whole project so knowledge is shared rather than split. Validation rules written once and shared. An offline AI engine built from the start so development never depends on a paid account and tests are repeatable.

Actions

- Babatundji: set up the repository and configuration, today.
- Eric: extract the design tokens from the reference screens, today.
- Victor: research the licences for the exercise and food data before any import code is written.

Next meeting: Tuesday 28 July, 10:00, Discord. Sign-in demonstration and data ingestion status.

Meeting 2: Sign-in and data ingestion

Field	Detail
Date and time	Tuesday 28 July 2026, 10:00
Location	Discord
Present	All three members
Absent	None
Objective	Verify sign-in end to end, unblock content ingestion

Discussion and decisions

- 1. Demonstrated and accepted:** registration and sign-in with session renewal, and Telegram sign-in with the three test cases (valid, tampered, expired).
- 2. Victor reported** the exercise import complete, 828 records, with licence and author retained per record. Confirmed the food database must be kept in a separate area because of its different licence terms.
- 3. Safety decision recorded:** calorie targets must be raised to a safe floor with a visible notice to the user, rather than silently adjusted. Implemented and tested the same day.
- 4. Concern raised** by Eric: the onboarding flow was becoming long. Agreed to keep it multi-step with progress shown, rather than cutting the questions, because the target calculation needs them.

Actions

- Eric: build the onboarding wizard.
- Babatundji: build food logging.
- Victor: add the nutrition data cross-checks.

Next meeting: Wednesday 29 July, 10:00. Full nutrition loop demonstration.

Meeting 3: Nutrition loop and AI rules

Field	Detail
Date and time	Wednesday 29 July 2026, 10:00
Location	Discord
Present	All three members
Absent	None
Objective	Demonstrate the complete nutrition loop, agree the rules for the AI layer

Discussion and decisions

- 1. Demonstrated and accepted:** manual logging with the dashboard reconciling to the raw records, barcode lookup, and the photo to confirmation to log journey.
- 2. Architectural decision recorded:** the AI identifies, the code calculates. No arithmetic is ever performed by an AI model. This makes results identical for identical inputs, cheaper, and checkable by hand.
- 3. Safety decision recorded:** a meal photo can never create a log entry without the user pressing confirm. Human approval is a hard requirement, not a setting.
- 4. Quality decision recorded:** the safety evaluations are a gate, not a report. If a critical case fails, the change does not ship.

Actions

- Babatundji: coach chat and personal memory.
- Eric: polish the capture and results screens, then build the coach screen.

- Victor: build the privacy isolation test suite.

Next meeting: Thursday 30 July, 10:00. Coach, memory and training.

Meeting 4: Coach, memory and training

Field	Detail
Date and time	Thursday 30 July 2026, 10:00
Location	Discord
Present	All three members
Absent	None
Objective	Demonstrate assistant safety and personal memory, review training module

Discussion and decisions

- 1. Demonstrated and accepted:** the coach refusing unsafe questions with a supportive signpost, the memory approval flow, and what happens when a user withdraws consent.
- 2. Privacy decision recorded:** withdrawing consent stops the app using stored memory, but does not delete it, because a user may re-enable it. A separate explicit delete action removes it permanently. Both behaviours are covered by tests so the distinction cannot be lost later.
- 3. Victor presented** the 24-test privacy isolation suite, all passing.
- 4. Risk raised by Babatundji:** the team had not verified how the hosting platform handles stored files. Assigned to him to confirm before the next meeting. This turned out to be the most important item of the week.

Actions

- Babatundji: hosting research, then final hardening.
- Eric: progress screen and settings.
- Victor: consolidate verification evidence for the Progress Report.

Next meeting: Friday 31 July, 15:00. Progress review and assessment sign-off.

Meeting 5: Progress review and 80% assessment

Field	Detail
Date and time	Friday 31 July 2026, 15:00
Location	Discord
Present	All three members
Absent	None

Field	Detail
Objective	Verify the 80% claim against evidence, sign off the Assessment 1 documents

Discussion and decisions

- 1. Hosting finding presented.** The platform erases stored files on every publish, which would have destroyed all user data. Babatundji presented the rebuilt database-backed storage and demonstrated data surviving a restart. The team accepted the recorded trade-off of single-server hosting for now.
- 2. Security and privacy review.** Location data stripping on photos, the upgraded upload library, the Telegram-compatible security headers, the stricter startup checks and the clean shutdown were all reviewed and accepted.
- 3. Evidence reviewed:** 431 tests passing across 35 files, type check clean, safety gate passed, live end-to-end run captured. The team agreed the 80% status is supportable: every core user journey works, and the outstanding 20% is publishing plus the legal and trust pages.
- 4. Contribution review.** Each member walked the others through their attributed work, confirming everyone can demonstrate and explain their own contribution at the review.
- 5. Honesty check.** The team agreed that anything not demonstrable is reported as In progress, and that the outstanding items are listed openly in the Progress Report rather than omitted.

Actions

- Babatundji: cloud photo storage, first publish, connect the email service.
- Eric: accessibility pass and Telegram polish.
- Victor: formal test evidence pack and the in-app attribution page.

Next meeting: Monday 3 August 2026, 10:00. Deployment execution.

14. Design and Style Constants

Item	Value
Design system name	Modern Aquatic Wellness, under the AquaZero brand
Visual approach	Translucent cards over a soft animated background, with blur used sparingly on navigation and dialogs for performance
Where tokens live	apps/web/tailwind.config.js and apps/web/src/styles/index.css
Shared components	apps/web/src/components/ui/
Reference screens	16 in design/figma/, covering welcome, sign-in, dashboard, nutrition, capture, analysis, meal

Item	Value
	plan, recipe, workout library, workout detail, progress, log weight, coach, settings and notifications
Accessibility	Icon-only controls carry descriptive labels; motion is disabled for users who request reduced motion

15. Data Storage Design

Information is stored as documents grouped into named collections: users, profiles, logs, plans, content, food data (kept in two separate collections to respect differing licences), AI data, the credit ledger and the audit trail.

In production every document is saved into a single database table:

```
CREATE TABLE IF NOT EXISTS documents (  
  container TEXT NOT NULL,  
  id        TEXT NOT NULL,  
  doc       JSONB NOT NULL,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (container, id)  
);
```

In plain terms: every piece of information is stored as a labelled record, with the label saying which collection it belongs to and which item it is. The combination of those two must be unique, which is what stops the same record being created twice.

Passwords are stored only as an irreversible scrambled value. Session renewal passes are stored the same way, so even full database access does not allow anyone to impersonate a user. The credit ledger is add-only: balances are calculated by adding up the history rather than by editing a stored number, so the record of what happened cannot be quietly altered.